



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

گزارش کار آزمایش هفتم

عنوان:

طراحی و پیاده‌سازی پروتکل UART

نام درس

آزمایشگاه طراحی سامانه‌های رقمی

تابستان ۱۴۰۵

استاد

دکتر اجلالی

دستیار استاد

علی اثنی عشری

گروه ۲

علی سلطانی - ۴۰۳۱۰۶۰۹۲

یگانه کربلایی - ۴۰۳۱۰۶۵۲۷

محمد مهدی مرادی - ۴۰۳۱۰۶۶۸۱

فهرست مطالب

۱	هدف آزمایش	۳
۲	شرح مسئله و قالب قاب UART	۳
۱.۲	ساختار قاب	۳
۲.۲	زمان بندی انتقال	۴
۳	طراحی فرستنده	۴
۱.۳	ساختار ماشین حالت فرستنده	۴
۲.۳	کد Verilog فرستنده	۵
۴	طراحی گیرنده	۱۰
۱.۴	ساختار ماشین حالت گیرنده	۱۰
۲.۴	کد Verilog گیرنده	۱۱
۵	پیمانه سطح بالا و اتصال بازگشت مستقیم	۱۵
۱.۵	کد Verilog پیمانه سطح بالا	۱۵
۶	سنتز و مشاهده ساختار در Quartus	۱۷
۱.۶	روند انجام کار در Quartus	۱۷
۲.۶	نمای RTL فرستنده	۱۸
۳.۶	نمای RTL گیرنده	۱۹
۴.۶	نمای RTL پیمانه سطح بالا	۲۰
۷	طراحی فایل آزمون	۲۰
۱.۷	هدف و ساختار فایل Testbench	۲۰
۲.۷	تولید ساعت، بازنشانی و ثبت شکل موج	۲۱
۳.۷	وظایف کمکی و خودآزمایی	۲۱
۸	سناریوهای آزمون و حالت های مرزی	۲۱
۱.۸	آزمون ۱: بازنشانی و حالت بی کار	۲۱
۲.۸	آزمون ۲: دریافت صحیح، قالب قاب و زمان بندی سیگنال busy	۲۲
۳.۸	آزمون ۳: خطای توازن	۲۲
۴.۸	آزمون ۴: خطای بیت توقف	۲۲
۵.۸	آزمون ۵: ارسال چند قاب پشت سرهم	۲۲
۶.۸	آزمون ۶: حداقل داده	۲۲
۷.۸	آزمون ۷: حداکثر داده	۲۳
۸.۸	آزمون ۸: فعال سازی new_data هنگام مشغول بودن فرستنده	۲۳
۹.۸	آزمون ۹: بازنشانی نهم زمان در میانه انتقال و بازیابی	۲۳
۱۰.۸	آزمون ۱۰: رد شروع کاذب و بازیابی گیرنده	۲۳
۱۱.۸	آزمون ۱۱: پیمایش جامع تمام ۱۲۸ مقدار داده	۲۳
۹	شکل موج های منتخب و تحلیل آن ها	۲۴

۲۴		test_id=2، دریافت صحیح،	۱.۹ شکل موج آزمون
۲۵		test_id=3 parity،	۲.۹ شکل موج آزمون خطای
۲۵		test_id=4،	۳.۹ شکل موج آزمون خطای بیت توقف،
۲۶		test_id=5،	۴.۹ شکل موج آزمون ارسال چند قاب،
۲۷		test_id=8، busy هنگام	۵.۹ شکل موج آزمون نادیده گرفتن new_data
۲۸		test_id=9،	۶.۹ شکل موج آزمون بازنشانی میانه انتقال،
۲۹		توضیح کلی	۷.۹ شکل موج ها
۲۹		خروجی نهایی	۱۰ فایل آزمون
۳۰		تحلیل نتایج و	۱۱ جمع بندی
۳۱		فایل های	۱۲ تحویلی

۱ هدف آزمایش

هدف این آزمایش، طراحی و پیاده‌سازی یک واحد ارتباط سریال ناهم‌زمان UART^۱ و بررسی عملکرد آن در سطح شبیه‌سازی است. در این طراحی، داده‌های هفت‌بیتی به‌صورت سریال و بر اساس یک قالب قاب مشخص منتقل می‌شوند. هر قاب شامل یک بیت شروع^۲، یک بیت توازن^۳، هفت بیت داده و یک بیت توقف^۴ است؛ در نتیجه برای هر داده، در مجموع ۱۰ بیت روی خط سریال قرار می‌گیرد.

هدف اصلی پیاده‌سازی، ساخت یک فرستنده، یک گیرنده و یک پیمانه سطح بالا است که خروجی سریال فرستنده را مستقیماً به ورودی سریال گیرنده متصل کند. به این ساختار، بازگشت مستقیم^۵ گفته می‌شود. در حالت صحیح، داده دریافت‌شده باید با داده ارسال‌شده برابر باشد و سیگنال صحت داده تنها در صورت معتبر بودن قاب فعال شود.

علاوه بر مسیر عادی، فایل آزمون به‌صورت جامع طراحی شده است تا خطاهای توازن و بیت توقف، درخواست داده در زمان مشغول‌بودن فرستنده، بازنشانی ناهم‌زمان در میانه انتقال، پالس شروع کاذب و همچنین تمامی ۱۲۸ مقدار ممکن داده هفت‌بیتی را بررسی کند. به این ترتیب، درستی مدار تنها با یک نمونه عادی اثبات نمی‌شود و حالت‌های مرزی نیز به‌صورت خودکار آزموده می‌شوند.

۲ شرح مسئله و قالب قاب UART

۱.۲ ساختار قاب

در حالت بی‌کار، خط سریال مقدار ۱ دارد. هنگام آغاز انتقال، فرستنده ابتدا یک صفر را به‌عنوان بیت شروع روی خط قرار می‌دهد. پس از آن بیت توازن، سپس بیت‌های داده از D0 تا D6 و در پایان بیت توقف با مقدار ۱ قرار می‌گیرند. ترتیب کامل قاب به‌صورت زیر است:

Idle (1) → Start (0) → Parity → D0 → D1 → D2 → D3 → D4 → D5 →
D6 → Stop (1)

بیت توازن از طریق عملگر XOR روی هفت بیت داده محاسبه می‌شود:

$$P = D_0 \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6$$

برای نمونه، در صورت ارسال داده

send_data = 7'b1011001

گیرنده باید پس از پایان قاب مقدار rec_data=1011001 را تولید کند و در صورت صحیح‌بودن بیت توازن و بیت توقف، سیگنال correct_data را برابر ۱ قرار دهد.

¹Universal Asynchronous Receiver Transmitter

²Start Bit

³Parity Bit

⁴Stop Bit

⁵Loopback

۲.۲ زمان‌بندی انتقال

نرخ استاندارد ذکر شده در صورت آزمایش برابر 115200 bps است. در این نرخ، زمان یک بیت از رابطه

$$T_{bit} = \frac{1}{115200}$$

به دست می‌آید. اگر ساعت سامانه ۵۰ مگاهرتز باشد، تعداد چرخه‌های ساعت لازم برای یک بیت تقریباً برابر است با

$$\frac{50\,000\,000}{115200} \simeq 434.$$

از آنجا که شبیه‌سازی با هدف مشاهده سریع‌تر شکل موج انجام شده است، در پیاده‌سازی حاضر زمان هر بیت با پارامتر BIT_TICKS کنترل می‌شود و مقدار آن برابر ۴ قرار گرفته است. همچنین دوره ساعت در فایل آزمون برابر ۱۰ نانوثانیه انتخاب شده است؛ در نتیجه هر بیت چهار چرخه ساعت، یعنی ۴۰ نانوثانیه، روی خط باقی می‌ماند.

۳ طراحی فرستنده

۱.۳ ساختار ماشین حالت فرستنده

فرستنده به صورت یک ماشین حالت متناهی^۶ پنج‌حالت پیاده‌سازی شده است. دلیل استفاده از ماشین حالت این است که ارسال یک قاب UART را می‌توان به چند مرحله متوالی و کاملاً مشخص تقسیم کرد و در هر مرحله مقدار خروجی، شمارنده زمان و سیگنال‌های کنترلی را به صورت مستقل کنترل نمود. در پیاده‌سازی حاضر این حالت‌ها شامل IDLE، START، PARITY، DATA و STOP هستند.

در حالت IDLE، خط tx در سطح منطقی ۱ نگه داشته می‌شود و busy برابر صفر است. در همین حالت، فرستنده منتظر یک پالس new_data می‌ماند. به محض مشاهده این سیگنال، مقدار هفت‌بیتی send_data در ثبات داخلی data_reg لچ می‌شود. این نکته اهمیت زیادی دارد، زیرا از این لحظه به بعد تغییرات روی ورودی خارجی send_data نباید روی قاب جاری اثر بگذارند. هم‌زمان بیت توازن با عملگر کاهش‌دهنده XOR روی همان داده محاسبه و در parity_reg ذخیره می‌شود. سپس busy فعال شده و خروجی tx مستقیماً به صفر تغییر می‌کند تا بیت شروع آغاز شود.

در حالت START مقدار صفر به مدت دقیقاً BIT_TICKS چرخه روی خط باقی می‌ماند. شمارنده tick_count در هر لبه بالارونده ساعت یک واحد افزایش پیدا می‌کند و هنگامی که به BIT_TICKS-1 برسد، زمان این بیت کامل شده و شمارنده دوباره صفر می‌شود. در همان لحظه فرستنده وارد حالت PARITY می‌شود و بیت توازن ذخیره شده را روی tx قرار می‌دهد. به این ترتیب مرز بین بیت شروع و بیت توازن کاملاً به زمان‌بندی ساعت وابسته و ثابت است.

در حالت PARITY، مقدار parity_reg برای یک زمان بیت کامل روی خط باقی می‌ماند. پس از پایان این زمان، اندیس بیت bit_idx صفر می‌شود و نخستین بیت داده یعنی data_reg[0] روی tx قرار می‌گیرد. این انتخاب عمداً با ترتیب صورت آزمایش هماهنگ است که بیت‌های داده را از کم‌ارزش‌ترین بیت، یعنی D0، به سوی پرارزش‌ترین بیت، یعنی D6، ارسال می‌کند.

حالت DATA بخش تکرارشونده ماشین حالت است. در هر مرحله، بیت فعلی با عبارت data_reg[bit_idx]

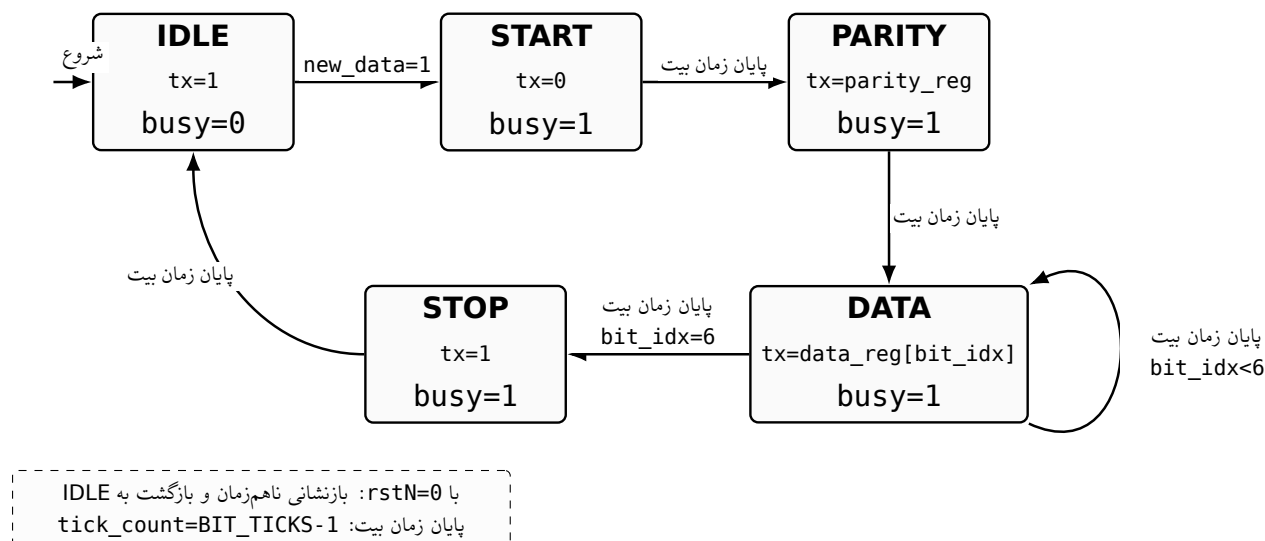
^۶Finite State Machine

انتخاب می‌شود و تا پایان BIT_TICKS چرخه روی خط باقی می‌ماند. اگر اندیس هنوز به ۶ نرسیده باشد، پس از پایان زمان بیت، bit_idx یک واحد افزایش می‌یابد و بیت بعدی انتخاب می‌شود. هنگامی که bit_idx=6 باشد، پس از پایان زمان D6، فرستنده به حالت STOP منتقل می‌شود و مقدار tx برابر یک قرار می‌گیرد.

در حالت STOP نیز مقدار یک به مدت یک زمان بیت روی خط باقی می‌ماند. این بیت پایان قاب را مشخص می‌کند و تا پایان آن، busy همچنان یک باقی می‌ماند. تنها پس از کامل شدن آخرین زمان بیت است که busy صفر می‌شود و ماشین حالت به IDLE بازمی‌گردد. بنابراین از دید مدار بالادست، صفر شدن busy به معنای پایان کامل قاب است و نه صرفاً پایان بخش داده.

یکی دیگر از ویژگی‌های مهم طراحی، بازنشانی ناهم‌زمان است. با فعال شدن rstN=0، حالت به IDLE، خروجی tx به ۱، busy به صفر و شمارنده‌ها به مقدار اولیه بازمی‌گردند. به همین دلیل حتی اگر بازنشانی در وسط یک قاب اعمال شود، فرستنده بدون انتظار برای لبه ساعت بعدی قاب را رها کرده و بلافاصله به وضعیت بی‌کار می‌رود. این ویژگی مستقیماً در آزمون نهم فایل آزمون نیز بررسی شده است.

همچنین، در حالت IDLE تنها زمانی داده جدید نمونه‌برداری می‌شود که new_data فعال باشد. از آنجا که در حالت‌های START تا STOP چنین شرطی وجود ندارد، درخواست مجدد new_data در زمان busy نمی‌تواند قاب جاری را تغییر دهد. این تصمیم طراحی در آزمون هشتم به‌طور مستقیم ارزیابی شده است.



شکل ۱: نمودار ماشین حالت فرستنده UART

۲.۳ کد Verilog فرستنده

کد کامل پیمانه فرستنده در فایل UARTSender.v قرار دارد:

```

1 module UARTSender (
2     input wire      clk,
3     input wire      rstN,
4     input wire      new_data,
5     input wire [6:0] send_data,
6
7     output reg tx,
8     output reg busy
9 );
    
```

```

10
11     parameter BIT_TICKS = 4;
12
13     // Sender states
14     localparam IDLE      = 3'b000;
15     localparam START     = 3'b001;
16     localparam PARITY    = 3'b010;
17     localparam DATA     = 3'b011;
18     localparam STOP      = 3'b100;
19
20     // Current state
21     reg [2:0] state;
22
23     // Data bit index
24     reg [2:0] bit_idx;
25
26     // Saved input data
27     reg [6:0] data_reg;
28
29     // Saved parity bit
30     reg      parity_reg;
31
32     // Bit timing counter
33     integer tick_count;
34
35     // Main sequential block
36     always @(posedge clk or negedge rstN)
37     begin
38         if (rstN == 1'b0)
39         begin
40             // Reset sender
41             state      <= IDLE;
42             tx          <= 1'b1;
43             busy        <= 1'b0;
44             bit_idx     <= 3'd0;
45             data_reg    <= 7'd0;
46             parity_reg  <= 1'b0;
47             tick_count  <= 0;
48         end
49         else
50         begin
51             case (state)
52
53                 IDLE:
54                 begin
55                     // Keep line idle high
56                     tx          <= 1'b1;

```

```

57
58     // Sender is idle
59     busy      <= 1'b0;
60
61     // Clear frame counters
62     bit_idx    <= 3'd0;
63     tick_count <= 0;
64
65     // Start on new data
66     if (new_data)
67     begin
68         // Save input data
69         data_reg <= send_data;
70
71         // Compute parity
72         parity_reg <= ^send_data;
73
74         // Start transmission
75         busy      <= 1'b1;
76         tx        <= 1'b0;
77         state     <= START;
78     end
79 end
80
81 START:
82 begin
83     // Send start bit
84     busy <= 1'b1;
85     tx   <= 1'b0;
86
87     // Keep start bit
88     if (tick_count == BIT_TICKS - 1)
89     begin
90         // Go to parity
91         tick_count <= 0;
92         tx          <= parity_reg;
93         state       <= PARITY;
94     end
95     else
96     begin
97         tick_count <= tick_count + 1;
98     end
99 end
100
101 PARITY:
102 begin
103     // Send parity bit

```

```

104         busy <= 1'b1;
105         tx      <= parity_reg;
106
107         // Keep parity bit
108         if (tick_count == BIT_TICKS - 1)
109             begin
110                 // Start with D0
111                 tick_count <= 0;
112                 bit_idx    <= 3'd0;
113                 tx          <= data_reg[0];
114                 state       <= DATA;
115             end
116         else
117             begin
118                 tick_count <= tick_count + 1;
119             end
120         end
121
122     DATA:
123     begin
124         // Send current bit
125         busy <= 1'b1;
126         tx      <= data_reg[bit_idx];
127
128         // Keep each data bit
129         if (tick_count == BIT_TICKS - 1)
130             begin
131                 tick_count <= 0;
132
133                 // Finish after D6
134                 if (bit_idx == 3'd6)
135                     begin
136                         tx          <= 1'b1;
137                         state <= STOP;
138                     end
139                 else
140                     begin
141                         // Select next bit
142                         bit_idx    <= bit_idx + 3'd1;
143                         tx          <= data_reg[bit_idx + 3'd1];
144                     end
145                 end
146             end
147         else
148             begin
149                 tick_count <= tick_count + 1;
150             end
151         end
152     end

```

```

151
152     STOP:
153     begin
154         // Send stop bit
155         busy  <= 1'b1;
156         tx    <= 1'b1;
157
158         // Keep stop bit
159         if (tick_count == BIT_TICKS - 1)
160         begin
161             // Frame complete
162             tick_count <= 0;
163             busy      <= 1'b0;
164             state     <= IDLE;
165         end
166         else
167         begin
168             tick_count <= tick_count + 1;
169         end
170     end
171
172     default:
173     begin
174         // Recover to idle
175         state <= IDLE;
176         tx    <= 1'b1;
177         busy  <= 1'b0;
178         bit_idx <= 3'd0;
179         tick_count <= 0;
180     end
181
182     endcase
183 end
184 end
185
186 endmodule

```

کد ۱: پیاده‌سازی فرستنده UART

در کد، چهار جزء اصلی مسئول تولید قاب هستند: ثابت `data_reg` برای نگهداری دادهٔ لچ‌شده، ثابت `parity_reg` برای بیت توازن، شمارندهٔ `tick_count` برای زمان‌بندی و `bit_idx` برای انتخاب بیت جاری. خروجی `tx` در هر حالت مستقیماً از وضعیت قاب تعیین می‌شود و `busy` نیز در تمام حالات غیر از `IDLE` فعال باقی می‌ماند. بنابراین ماشین حالت علاوه بر کنترل ترتیب منطقی قاب، کنترل زمانی انتقال را نیز بر عهده دارد.

۴ طراحی گیرنده

۱.۴ ساختار ماشین حالت گیرنده

گیرنده نیز با یک ماشین حالت متناهی پنج‌حالت پیاده‌سازی شده است که همان ساختار کلی قاب فرستنده را دنبال می‌کند؛ با این تفاوت که وظیفه آن نمونه‌برداری از خط ورودی و بازسازی تدریجی قاب است. حالت‌های اصلی عبارت‌اند از IDLE، DATA، PARITY، START و STOP. علاوه بر ثبات حالت، گیرنده از ثبات data_reg برای ذخیره داده دریافتی، parity_reg برای نگهداری parity دریافت‌شده، bit_idx برای مشخص کردن شماره بیت و tick_count برای کنترل زمان نمونه‌برداری استفاده می‌کند.

در حالت IDLE خط rx پیوسته بررسی می‌شود و تا زمانی که مقدار آن ۱ باشد، گیرنده در همین حالت باقی می‌ماند. مشاهده یک صفر نخستین نشانه شروع قاب است و باعث می‌شود ماشین وارد حالت START شود. نکته مهم این است که این صفر به‌تنهایی به‌عنوان start معتبر پذیرفته نمی‌شود. گیرنده باید پس از گذشت نیم زمان بیت دوباره خط را بررسی کند تا مشخص شود سطح صفر واقعاً مربوط به بیت شروع بوده است. برای این منظور پارامتر محلی HALF_BIT_TICKS از BIT_TICKS به‌دست می‌آید که در پیاده‌سازی حاضر برابر دو چرخه است.

این تأیید میانی نقش مهمی در حذف پالس‌های کاذب دارد. اگر ورودی فقط برای مدت کوتاهی صفر شود و پیش از رسیدن به مرکز بیت دوباره یک شود، شرط حالت START برقرار نمی‌شود و ماشین مستقیماً به IDLE بازمی‌گردد. در نتیجه هیچ parity داده یا bit stop اشتباهی دریافت نمی‌شود. همین رفتار در آزمون دهم، یعنی آزمون false start، مستقیماً بررسی شده است.

پس از تأیید start، گیرنده وارد حالت PARITY می‌شود. در این مرحله یک زمان بیت کامل صبر می‌کند و سپس مقدار rx را در parity_reg ذخیره می‌کند. پس از ذخیره parity، اندیس bit_idx از صفر شروع می‌شود تا دریافت هفت بیت داده را کنترل کند.

در حالت DATA، در پایان هر زمان بیت مقدار فعلی rx در خانه متناظر data_reg[bit_idx] نوشته می‌شود. بنابراین اولین نمونه در data_reg[0] و آخرین نمونه در data_reg[6] قرار می‌گیرد. این ترتیب دقیقاً با ترتیب ارسال فرستنده سازگار است و باعث می‌شود پس از دریافت آخرین بیت، data_reg همان مقدار هفت‌بیتی ارسال‌شده را در خود داشته باشد. اگر اندیس کمتر از ۶ باشد، تنها اندیس افزایش پیدا می‌کند و دریافت ادامه می‌یابد. پس از دریافت D6، ماشین به حالت STOP منتقل می‌شود.

در حالت STOP گیرنده پس از پایان زمان بیت، ابتدا مقدار خط را به‌عنوان bit stop بررسی می‌کند. اگر rx=1 باشد، شرط نخست اعتبار قاب برقرار است. هم‌زمان parity محاسبه‌شده از هفت بیت داده با مقدار ذخیره‌شده در parity_reg مقایسه می‌شود. شرط نهایی در واقع ترکیب این دو بررسی است:

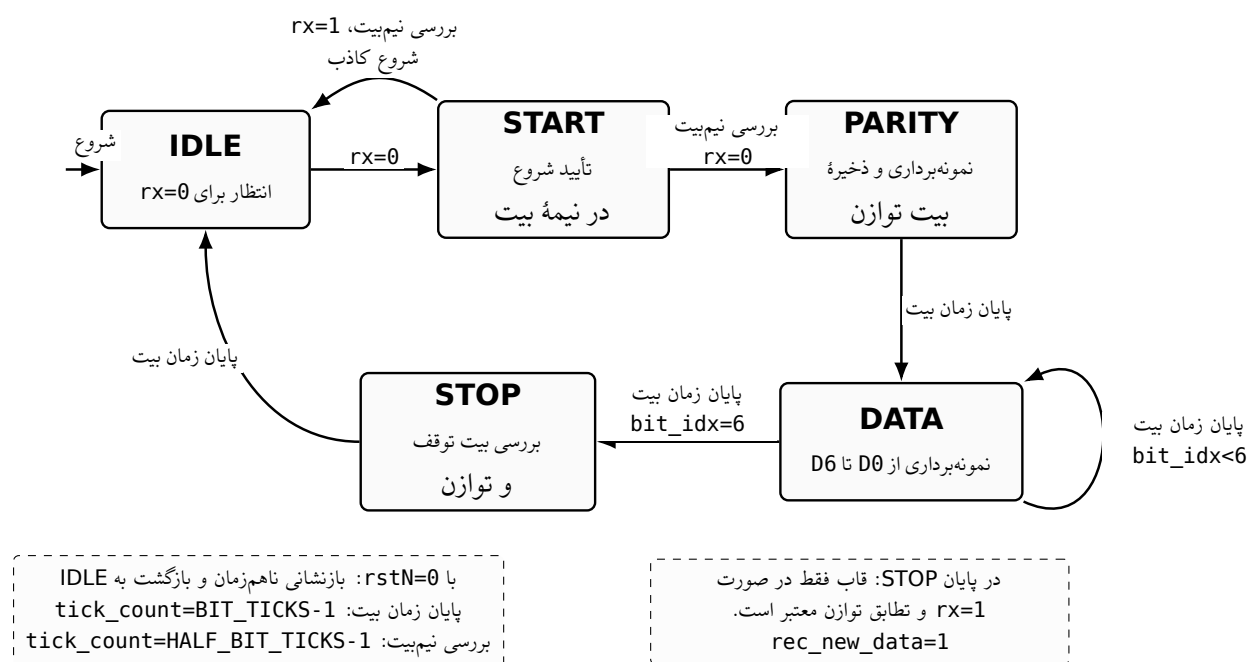
$$(rx = 1) \quad \wedge \quad (\oplus data_reg = parity_reg)$$

تنها در صورت برقرار بودن هر دو شرط، correct_data برابر یک می‌شود؛ در غیر این صورت مقدار آن صفر خواهد بود. بنابراین قاب دارای parity اشتباه یا bit stop صفر، حتی اگر داده خام آن دریافت شده باشد، به‌عنوان قاب صحیح گزارش نمی‌شود.

صرف‌نظر از معتبر یا نامعتبر بودن قاب، داده موجود در data_reg در پایان حالت STOP به rec_data منتقل می‌شود و rec_new_data برای یک چرخه فعال می‌گردد. این جداسازی مهم است: rec_new_data اعلام می‌کند که فرایند دریافت یک قاب تمام شده است، درحالی‌که correct_data مشخص می‌کند آیا آن قاب از نظر شرایط صحت معتبر بوده یا خیر. بنابراین مصرف‌کننده مدار می‌تواند بین «داده دریافت شده» و «داده معتبر» تمایز بگذارد.

پس از پایان STOP، گیرنده دوباره به IDLE بازمی‌گردد و برای قاب بعدی آماده می‌شود. همچنین در ابتدای هر چرخه عادی، `rec_new_data` به صورت پیش فرض صفر می‌شود؛ به همین دلیل این سیگنال ذاتاً یک پالس یک‌کلاکی است و در آزمون‌های فایل آزمون نیز این ویژگی به صورت مستقل بررسی شده است.

بازنشانی گیرنده نیز فعال‌پایین و ناهم‌زمان است. با اعمال `rstN=0`، حالت، داده، `parity` اندیس بیت و شمارنده زمان همگی به مقادیر اولیه بازمی‌گردند و `rec_new_data` و `correct_data` فوراً صفر می‌شوند. در نتیجه اگر بازنشانی وسط دریافت اتفاق بیفتد، قاب ناقص نمی‌تواند به عنوان یک دریافت کامل یا صحیح ثبت شود. این رفتار در آزمون نهم به صورت مستقیم ارزیابی شده است.



شکل ۲: نمودار ماشین حالت گیرنده UART

۲.۴ کد Verilog گیرنده

کد کامل پیمانه گیرنده در فایل `UARTReceiver.v` آمده است:

```

1 module UARTReceiver (
2     input wire      clk,
3     input wire      rstN,
4     input wire      rx,
5
6     output reg [6:0] rec_data,
7     output reg      rec_new_data,
8     output reg      correct_data
9 );
10
11 parameter BIT_TICKS = 4;
12
13 // Receiver states
14 localparam IDLE      = 3'b000;
15 localparam START     = 3'b001;
  
```

```

16  localparam PARITY = 3'b010;
17  localparam DATA   = 3'b011;
18  localparam STOP    = 3'b100;
19
20  localparam HALF_BIT_TICKS = (BIT_TICKS >= 2) ? (BIT_TICKS / 2) : 1;
21
22  // Current state
23  reg [2:0] state;
24
25  // Data bit index
26  reg [2:0] bit_idx;
27
28  // Received data buffer
29  reg [6:0] data_reg;
30
31  // Saved parity bit
32  reg parity_reg;
33
34  // Bit timing counter
35  integer tick_count;
36
37  // Main sequential block
38  always @(posedge clk or negedge rstN)
39  begin
40      if (rstN == 1'b0)
41      begin
42          // Reset receiver
43          state          <= IDLE;
44          rec_data        <= 7'b0;
45          rec_new_data    <= 1'b0;
46          correct_data    <= 1'b0;
47          bit_idx         <= 3'd0;
48          data_reg        <= 7'd0;
49          parity_reg      <= 1'b0;
50          tick_count      <= 0;
51      end
52      else
53      begin
54          // Default pulse low
55          rec_new_data    <= 1'b0;
56
57          case (state)
58
59              IDLE:
60              begin
61                  // Wait for start
62                  tick_count <= 0;

```

```

63         bit_idx      <= 3'd0;
64
65         // Detect start bit
66         if (rx == 1'b0)
67         begin
68             correct_data    <= 1'b0;
69             state           <= START;
70         end
71     end
72
73     START:
74     begin
75         // Check start near center
76         if (tick_count == HALF_BIT_TICKS - 1)
77         begin
78             tick_count <= 0;
79
80             if (rx == 1'b0)
81             begin
82                 state <= PARITY;
83             end
84             else
85             begin
86                 state <= IDLE;
87             end
88         end
89         else
90         begin
91             tick_count <= tick_count + 1;
92         end
93     end
94
95     PARITY:
96     begin
97         // Sample parity bit
98         if (tick_count == BIT_TICKS - 1)
99         begin
100             tick_count    <= 0;
101             parity_reg     <= rx;
102             bit_idx       <= 3'd0;
103             state         <= DATA;
104         end
105         else
106         begin
107             tick_count <= tick_count + 1;
108         end
109     end

```

```

110
111     DATA:
112     begin
113         // Sample D0 to D6
114         if (tick_count == BIT_TICKS - 1)
115         begin
116             tick_count      <= 0;
117             data_reg[bit_idx] <= rx;
118
119             // Move to stop bit
120             if (bit_idx == 3'd6)
121             begin
122                 state <= STOP;
123             end
124             else
125             begin
126                 bit_idx <= bit_idx + 3'd1;
127             end
128         end
129         else
130         begin
131             tick_count <= tick_count + 1;
132         end
133     end
134
135     STOP:
136     begin
137         // Check stop bit
138         if (tick_count == BIT_TICKS - 1)
139         begin
140             tick_count      <= 0;
141             rec_data         <= data_reg;
142             rec_new_data     <= 1'b1;
143
144             // Validate frame
145             if ((rx == 1'b1) && ((^data_reg) == parity_reg))
146             begin
147                 correct_data <= 1'b1;
148             end
149             else
150             begin
151                 correct_data <= 1'b0;
152             end
153
154             // Return to idle
155             state <= IDLE;
156         end

```

```

157         else
158         begin
159             tick_count <= tick_count + 1;
160         end
161     end
162
163     default:
164     begin
165         // Recover to idle
166         state          <= IDLE;
167         rec_new_data    <= 1'b0;
168         correct_data    <= 1'b0;
169         bit_idx         <= 3'd0;
170         tick_count      <= 0;
171     end
172
173 endcase
174 end
175 end
176
177 endmodule

```

کد ۲: پیاده‌سازی گیرنده UART

در این کد، HALF_BIT_TICKS برای تأیید مرکز، tick_count برای نگهداری زمان هر بیت، bit_idx برای ترتیب ذخیره هفت بیت و parity_reg برای نگهداری parity دریافتی به کار می‌روند. در نتیجه کل فرآیند دریافت، از تشخیص شروع تا بررسی نهایی قاب، تنها با یک ماشین حالت متناهی و مجموعه‌ای کوچک از ثبات‌های زمان‌بندی و ذخیره‌سازی کنترل می‌شود.

۵ پیمانه سطح بالا و اتصال بازگشت مستقیم

برای آزمایش مسیر کامل انتقال، دو پیمانه فرستنده و گیرنده در یک پیمانه سطح بالا به نام UARTTop.v نمونه‌سازی شده‌اند. خروجی سریال فرستنده در سیم داخلی rx_line قرار می‌گیرد و همان سیم بدون هیچ عنصر میانجی به ورودی rx گیرنده متصل می‌شود. بنابراین ساختار بازگشت مستقیم ایجاد شده است.

این اتصال دو مزیت مهم دارد: نخست آنکه نیازی به تولید یک مسیر سیمی خارجی برای آزمایش دریافت وجود ندارد و دوم آنکه کل زنجیره ارسال و دریافت به‌صورت یک واحد واحد قابل آزمون است. در حالت صحیح، هر داده‌ای که فرستنده ارسال کند باید در خروجی گیرنده همان داده را تولید کند.

۱.۵ کد Verilog پیمانه سطح بالا

کد کامل پیمانه سطح بالا:

```

1 module UARTTop (
2     input wire      clk,
3     input wire      rstN,
4     input wire      new_data,
5     input wire [6:0] send_data,
6
7     output wire      tx,
8     output wire      busy,
9     output wire      rec_new_data,
10    output wire [6:0] rec_data,
11    output wire      correct_data
12 );
13
14 parameter BIT_TICKS = 4;
15
16 // Loopback line
17 wire rx_line;
18
19 // Expose serial line
20 assign tx = rx_line;
21
22 // Sender instance
23 UARTSender #(
24     .BIT_TICKS(BIT_TICKS)
25 ) sender_inst (
26     .clk      (clk),
27     .rstN     (rstN),
28     .new_data (new_data),
29     .send_data (send_data),
30
31     .tx      (rx_line),
32     .busy    (busy)
33 );
34
35 // Receiver instance
36 UARTReceiver #(
37     .BIT_TICKS(BIT_TICKS)
38 ) receiver_inst (
39     .clk      (clk),
40     .rstN     (rstN),
41     .rx       (rx_line),
42
43     .rec_data (rec_data),
44     .rec_new_data (rec_new_data),
45     .correct_data (correct_data)
46 );

```

کد ۳: اتصال فرستنده و گیرنده در پیمانه UARTTop

۶ سنتز و مشاهده ساختار در Quartus

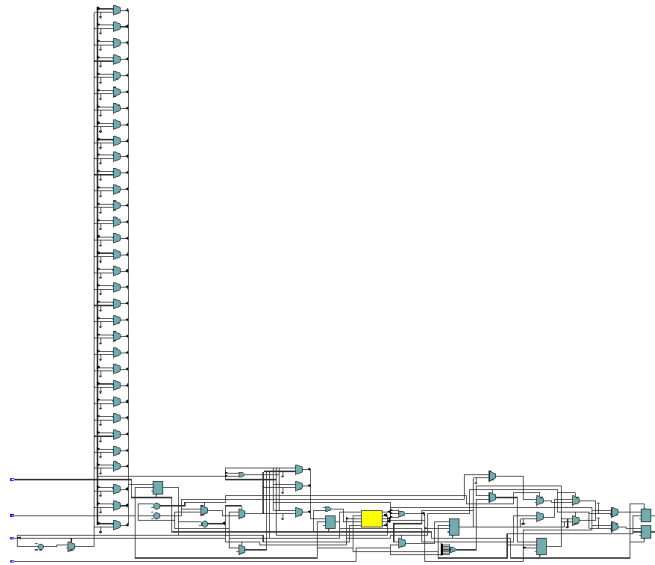
۱.۶ روند انجام کار در Quartus

برای مشاهده ساختار سخت‌افزاری حاصل از کدهای Verilog، ابتدا پروژه در Quartus ایجاد شد و فایل‌های UARTSender.v، UARTReceiver.v و UARTTop.v به پروژه افزوده شدند. سپس پیمانه UARTTop به‌عنوان موجودیت سطح بالا انتخاب شد. پس از آن تحلیل و سنتز پروژه انجام گرفت تا قابل سنتز بودن طراحی بررسی شود. مسیر مورد استفاده برای بازکردن نمای سطح انتقال ثبات^۷ به‌صورت زیر است:

Tools > Netlist Viewers > RTL Viewer

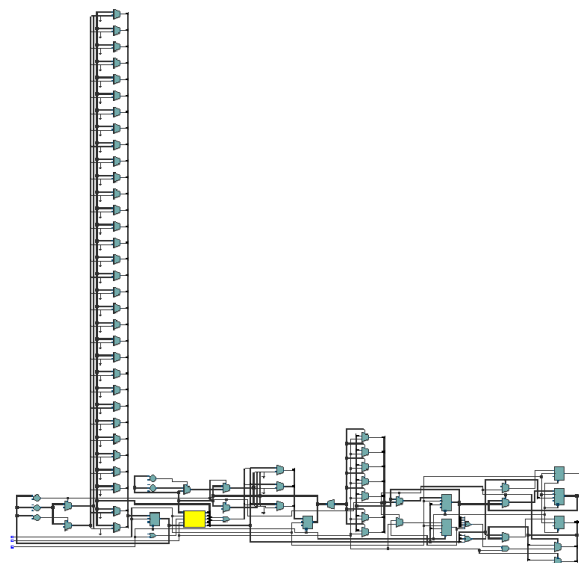
پس از اجرای این گزینه، Quartus ساختار منطقی استخراج‌شده از توصیف Verilog را نمایش می‌دهد. در این نما، ثبات‌ها، منطق انتخاب، شمارنده‌ها، مسیرهای داده و روابط بین آن‌ها قابل مشاهده هستند. در گزارش حاضر سه تصویر مستقل از ساختارهای تولیدشده برای پیمانه‌های اصلی آورده شده است.

⁷Register-Transfer Level (RTL)



شکل ۳: نمای RTL حاصل از سنتز پیمانه UARTSender

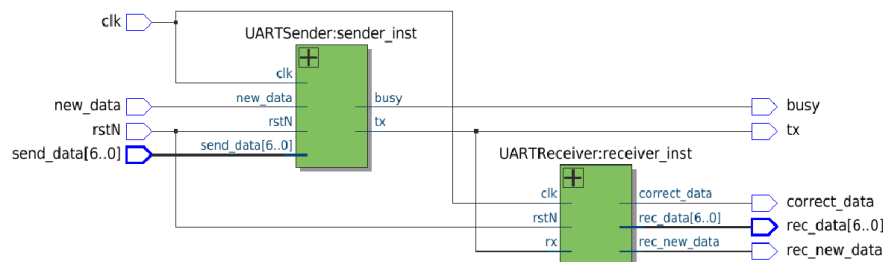
در شکل ۳، ساختار منطقی حاصل از توصیف رفتاری فرستنده دیده می‌شود. شلوغی این نما طبیعی است؛ زیرا ماشین حالت، شمارنده زمان‌بندی، ثبات داده، محاسبه توازن و منطق انتخاب بیت جاری پس از سنتز به مجموعه‌ای از عناصر منطقی صریح تبدیل شده‌اند. وجود مسیرهای متعدد در بخش پایینی شکل با تصمیم طراحی برای نگهداری حالت، شمارش چرخه‌ها و انتخاب `data_reg[bit_idx]` مرتبط است.



شکل ۴: نمای RTL حاصل از سنتز پیمانه UARTReceiver

شکل ۴ ساختار گیرنده را نشان می‌دهد. در این نما علاوه بر ثبات‌های مربوط به حالت و شمارنده زمان، مسیر مربوط به ذخیره داده و بیت توازن و همچنین منطق بررسی شرط نهایی قاب دیده می‌شود. پیچیدگی بیشتر ساختار گیرنده نسبت به یک منطق ترکیبی ساده، ناشی از دریافت مرحله به مرحله قاب و کنترل زمان نمونه برداری است.

۴.۶ نمای RTL پیمانه سطح بالا



شکل ۵: نمای RTL پیمانه سطح بالا و اتصال مستقیم فرستنده به گیرنده

شکل ۵ نمایی بسیار خواناتر از ساختار سطح بالا ارائه می‌کند. در سمت چپ ورودی‌های ساعت، بازنشانی، درخواست داده و داده ارسالی وارد فرستنده می‌شوند. خروجی سریال فرستنده به‌طور مستقیم به ورودی سریال گیرنده وصل شده است و در سمت راست نیز `rec_data`، `rec_new_data` و `correct_data` مشاهده می‌شوند. این شکل دقیقاً رابط اصلی‌ای را که در شبیه‌سازی آزموده شده است نمایش می‌دهد.

۷ طراحی فایل آزمون

۱.۷ هدف و ساختار فایل Testbench

فایل آزمون^۸ در `Tester.v` طراحی شده و تنها به یک سناریوی ساده ارسال و دریافت محدود نیست. در این فایل دو پیمانه تحت آزمون وجود دارد: نخست `top_dut` که کل مسیر فرستنده، بازگشت مستقیم و گیرنده را شامل می‌شود و دوم یک گیرنده مستقل به نام `manual_rx_dut`. گیرنده دوم عمداً به یک ورودی مستقل وصل شده است تا بتوان قاب‌های خراب را مستقیماً به آن اعمال کرد. به این ترتیب خطای توازن و خطای بیت توقف، بدون وابستگی به درست یا غلط بودن فرستنده، قابل آزمون هستند.

^۸Testbench

۲.۷ تولید ساعت، بازنشانی و ثبت شکل موج

در فایل آزمون، پارامتر $BIT_TICKS=4$ و دوره ساعت $CLK_PERIOD=10$ نانوثانیه تعیین شده‌اند. ساعت به صورت پیوسته با یک فرایند `forever` تولید می‌شود. بازنشانی فعال‌پایین و ناهم‌زمان است و تابع `reset_all` بلافاصله پس از اعمال صفر به `rstN`، وضعیت خروجی‌های مهم را بررسی می‌کند.

برای تولید فایل قابل مشاهده در نرم‌افزار تحلیل شکل موج، خروجی VCD^9 با نام `uart_tests.vcd` ساخته می‌شود. در این فایل فقط سیگنال‌های لازم برای گزارش ذخیره شده‌اند تا نمای حاصل خوانا باقی بماند. از جمله سیگنال‌های ذخیره شده می‌توان به `rstN`، `clk`، `send_data`، `tx`، `busy`، `rec_data`، `rec_new_data` و `correct_data` اشاره کرد. همچنین مقدار `test_id` عمداً در فایل شکل موج ذخیره شده است تا محدوده هر آزمون در تحلیل تصویری به وضوح قابل تشخیص باشد.

۳.۷ وظایف کمکی و خودآزمایی

برای جلوگیری از تکرار منطق آزمون، چند وظیفه کمکی در فایل تعریف شده است. مهم‌ترین آن‌ها عبارت‌اند از:

- `add_check`: یک شرط را ثبت می‌کند، شمارنده بررسی‌ها را افزایش می‌دهد و در صورت شکست، شمارنده خطا را نیز افزایش می‌دهد.
- `reset_all`: بازنشانی اولیه را اعمال کرده و وضعیت بی‌کار فرستنده و گیرنده را کنترل می‌کند.
- `send_top_data`: پس از اطمینان از غیرمشغول بودن فرستنده، داده را در ورودی قرار داده و یک پالس کوتاه `new_data` تولید می‌کند.
- `check_top_data`: منتظر پالس `rec_new_data` می‌ماند و سپس هم داده و هم `correct_data` را با مقدار مورد انتظار مقایسه می‌کند. یک کلاک بودن این پالس نیز بررسی می‌شود.
- `send_manual_frame`: یک قاب کامل را به صورت دستی روی ورودی گیرنده مستقل اعمال می‌کند تا بتوان مقدار `stop` و `parity` را عمداً تغییر داد.
- `check_serial_frame`: تک‌تک بیت‌های روی `tx` و همچنین فعال بودن `busy` را در تمام چرخه‌های هر بیت بررسی می‌کند؛ بنابراین یک تست صرفاً مبتنی بر `loopback` نمی‌تواند خطای مشترک فرستنده و گیرنده را پنهان کند.

۸ سناریوهای آزمون و حالت‌های مرزی

مجموعه آزمون شامل ۱۱ سناریوی اصلی است. این سناریوها فقط مسیر عادی ارسال و دریافت را بررسی نمی‌کنند، بلکه رفتار مدار را در برابر خطا، مرزهای داده، درخواست‌های نامعتبر و رخداد‌های غیرعادی نیز ارزیابی می‌کنند. در ادامه هر یک از آزمون‌ها به ترتیب اجرای فایل `Tester.v` توضیح داده می‌شود.

۱.۸ آزمون ۱: بازنشانی و حالت بی‌کار

در نخستین آزمون، پس از اعمال بازنشانی، وضعیت اولیه کل زنجیره انتقال و دریافت بررسی می‌شود. در حالت بازنشانی انتظار می‌رود فرستنده بلافاصله به حالت بی‌کار برگردد؛ بنابراین `tx` باید برابر ۱ و `busy` برابر صفر باشد. هم‌زمان، سیگنال‌های

⁹Value Change Dump

وضعیت گیرنده یعنی `rec_new_data` و `correct_data` نیز باید صفر باشند. این آزمون علاوه بر بررسی وضعیت بی‌کار، رفتار بازنشانی نهم‌زمان را نیز کنترل می‌کند؛ یعنی مدار نباید برای مشاهده لبه ساعت صبر کند تا اثر `rstN=0` اعمال شود.

۲.۸ آزمون ۲: دریافت صحیح، قالب قاب و زمان‌بندی سیگنال `busy`

در این آزمون داده هفت‌بیتی `1011001` از مسیر اصلی `UARTTop` ارسال می‌شود. فایل آزمون سه بررسی را به‌صورت هم‌زمان انجام می‌دهد: نخست داده به فرستنده داده می‌شود، دوم قالب کامل قاب مستقیماً روی `tx` کنترل می‌شود و سوم گیرنده منتظر `rec_new_data` می‌ماند تا داده نهایی را با مقدار مورد انتظار مقایسه کند. بنابراین آزمون فقط برابر بودن داده ارسال شده و دریافت شده را بررسی نمی‌کند، بلکه ترتیب بیت‌ها، بیت شروع، `parity` هفت بیت داده، بیت توقف و فعال‌بودن `busy` در تمام مدت قاب نیز کنترل می‌شوند. در پایان، `busy` باید تنها پس از پایان بیت توقف صفر شود و `tx` دوباره به وضعیت بی‌کار یعنی ۱ بازگردد.

۳.۸ آزمون ۳: خطای توازن

برای آزمون خطای `parity` از گیرنده مستقل `manual_rx_dut` استفاده شده است. داده همچنان `1011001` است، اما بیت توازن عمداً معکوس می‌شود. به این ترتیب داده دریافت شده می‌تواند در ثبات گیرنده ثبت شود، ولی شرط اعتبار قاب دیگر برقرار نیست. انتظار می‌رود `manual_correct_data` صفر شود. این ساختار آزمون باعث می‌شود تشخیص خطای `parity` مستقیماً در خود گیرنده سنجیده شود و نتیجه به عملکرد فرستنده اصلی وابسته نباشد.

۴.۸ آزمون ۴: خطای بیت توقف

در آزمون چهارم، `parity` مطابق مقدار صحیح داده تولید می‌شود، اما بیت توقف به‌طور عمدی صفر قرار داده می‌شود. بنابراین حتی در صورت درست بودن داده و `parity` قاب نباید معتبر اعلام شود. گیرنده در پایان قاب داده را ثبت می‌کند و رویداد دریافت را صادر می‌کند، ولی `manual_correct_data` باید برابر صفر باقی بماند. این آزمون نشان می‌دهد که شرط اعتبار قاب در پیاده‌سازی فقط به `parity` محدود نیست و مقدار بیت توقف نیز به‌طور مستقل بررسی می‌شود.

۵.۸ آزمون ۵: ارسال چند قاب پشت‌سرهم

در این سناریو دو داده متفاوت یعنی `1011001` و `0101101` در دو قاب جداگانه ارسال می‌شوند. فایل آزمون پس از پایان قاب اول منتظر غیرفعال شدن `busy` می‌ماند و سپس قاب دوم را آغاز می‌کند. در سمت دریافت نیز دو بار منتظر پالس `rec_new_data` می‌ماند و هر دو مقدار را جداگانه با داده مورد انتظار مقایسه می‌کند. هدف این آزمون اطمینان از آن است که پایان یک قاب و آغاز قاب بعدی به‌درستی از یکدیگر تفکیک شوند و یک رویداد دریافت به‌جای دو قاب ایجاد نشود.

۶.۸ آزمون ۶: حداقل داده

در این آزمون کمینه عددی فضای ورودی یعنی `0000000` ارسال می‌شود. برای این داده `parity` برابر صفر است، بنابراین علاوه بر بررسی مسیر معمول دریافت، حالت مرزی `parity=0` نیز آزمایش می‌شود. انتظار می‌رود `rec_data` برابر `0000000` و `correct_data` برابر یک شود. این سناریو ثابت می‌کند که مدار در مرز پایین فضای داده نیز بدون ایجاد حالت خاص یا خطای محاسباتی عمل می‌کند.

۷.۸ آزمون ۷: حداکثر داده

در آزمون هفتم بیشینه مقدار ممکن برای داده هفت بیتی یعنی 11111111 ارسال می‌شود. در این حالت parity برابر یک است. این آزمون مکمل آزمون ششم است و علاوه بر بررسی مرز بالای داده، مسیر معتبر $\text{parity}=1$ را نیز فعال می‌کند؛ در نتیجه دو مقدار مرزی و هر دو حالت اصلی parity در آزمون‌های جهت‌دار پوشش داده می‌شوند.

۸.۸ آزمون ۸: فعال‌سازی new_data هنگام مشغول بودن فرستنده

در آزمون هشتم ابتدا داده 0010110 برای آغاز یک قاب در فرستنده قرار می‌گیرد. پس از آن، در حالی که $\text{busy}=1$ است، مقدار ورودی به 1101001 تغییر داده شده و یک پالس جدید new_data اعمال می‌شود. رفتار مورد انتظار آن است که درخواست جدید نادیده گرفته شود و داده ذخیره‌شده قاب جاری تغییر نکند. فایل آزمون علاوه بر بررسی دریافت صحیح قاب اول، تعداد رخداد‌های دریافت را نیز می‌شمارد تا اطمینان حاصل شود که درخواست زمان مشغول بودن، قاب دوم اضافی ایجاد نکرده است. بنابراین این آزمون هم لچ شدن داده در آغاز انتقال و هم حفاظت از قاب جاری را بررسی می‌کند.

۹.۸ آزمون ۹: بازنشانی نهم‌زمان در میانه انتقال و بازیابی

در آزمون نهم یک قاب با داده 0110101 شروع می‌شود و پس از چند چرخه، بازنشانی نهم‌زمان در وسط قاب فعال می‌گردد. انتظار می‌رود فرستنده فوراً انتقال را قطع کرده و به وضعیت $\text{tx}=1$, $\text{busy}=0$ برگردد. در گیرنده نیز وضعیت‌های دریافت باید پاک شوند و قاب ناقص نباید به عنوان یک رویداد دریافت معتبر ثبت شود. پس از آزاد شدن بازنشانی، یک قاب جدید با داده 1000011 ارسال می‌شود تا توانایی مدار در بازیابی کامل پس از قطع ناگهانی نیز اثبات شود.

۱۰.۸ آزمون ۱۰: رد شروع کاذب و بازیابی گیرنده

در آزمون دهم ابتدا یک پالس کوتاه صفر روی manual_rx ایجاد می‌شود که طول آن از زمان لازم برای تأیید بیت شروع کمتر است. گیرنده باید این رخداد را یک شروع واقعی در نظر نگیرد و هیچ رویداد دریافت جدیدی تولید نکند. سپس یک قاب معتبر با داده 0000001 و parity صحیح ارسال می‌شود تا مشخص شود رد شدن شروع کاذب باعث قفل شدن گیرنده نشده و گیرنده می‌تواند بلافاصله به حالت عادی بازگردد. این آزمون مستقیماً منطق تأیید start در نزدیکی مرکز بیت را ارزیابی می‌کند.

۱۱.۸ آزمون ۱۱: پیمایش جامع تمام ۱۲۸ مقدار داده

در آخرین آزمون، تمام مقادیر ممکن یک داده هفت‌بیتی از صفر تا ۱۲۷ به صورت پشت‌سرهم ارسال می‌شوند. این آزمون از نظر پوشش فضای داده کامل است و به جای تکیه بر چند نمونه خاص، رفتار فرستنده و گیرنده را برای تمام ترکیب‌های ممکن داده بررسی می‌کند. برای جلوگیری از شلوغ شدن فایل شکل موج، ثبت VCD در طول این آزمون موقتاً غیرفعال می‌شود، اما بررسی‌های منطقی همچنان انجام می‌شوند. در پایان باید تمام ۱۲۸ مقدار بدون خطا دریافت شوند.

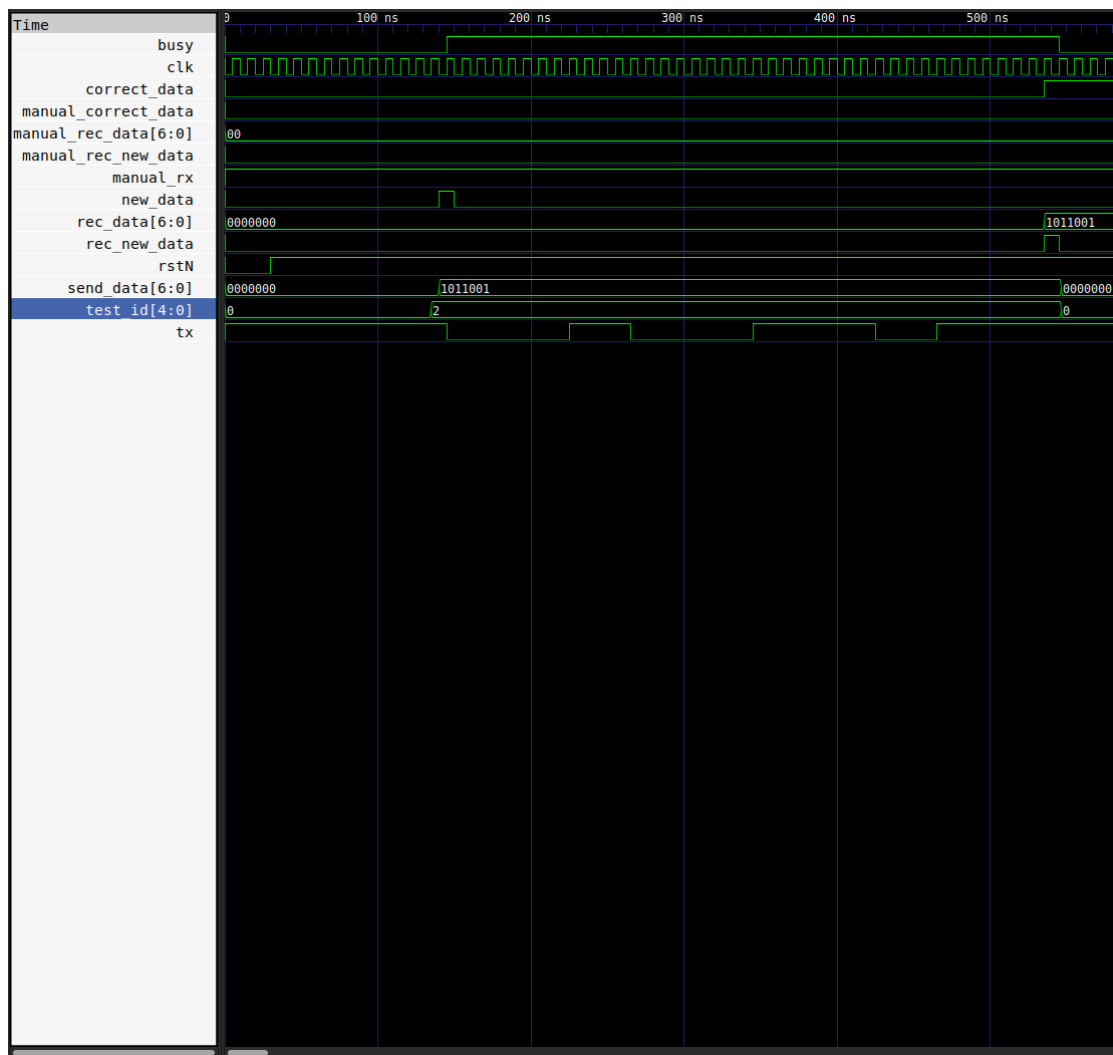
آزمون‌های بالا در مجموع مسیر صحیح، دو نوع خطای قاب، مرزهای داده، درخواست در زمان مشغول بودن، بازنشانی میانه قاب، شروع کاذب و کل فضای داده ورودی را پوشش می‌دهند. بنابراین اعتبار نتیجه نهایی تنها بر یک شکل موج یا یک داده نمونه متکی نیست.

۹ شکل موج‌های منتخب و تحلیل آن‌ها

از آنجا که اجرای کامل فایل آزمون شامل ۱۱ سناریو و در آزمون آخر ۱۲۸ قاب است، نمایش همه بازه‌های زمانی در یک شکل واحد باعث شلوغی شدید نمودار می‌شود. به همین دلیل در گزارش، شکل موج‌های منتخب مربوط به آزمون‌های مهم آورده شده‌اند. با این حال، پیش از نمایش این تصاویر، همه `test_id`ها و هدف آن‌ها در بخش قبل توضیح داده شده‌اند؛ بنابراین شکل‌های حاضر نمونه‌هایی از اجرای همان سناریوها هستند و جایگزین آزمون‌های اجرا شده نیستند.

۱.۹ شکل موج آزمون دریافت صحیح، `test_id=2`

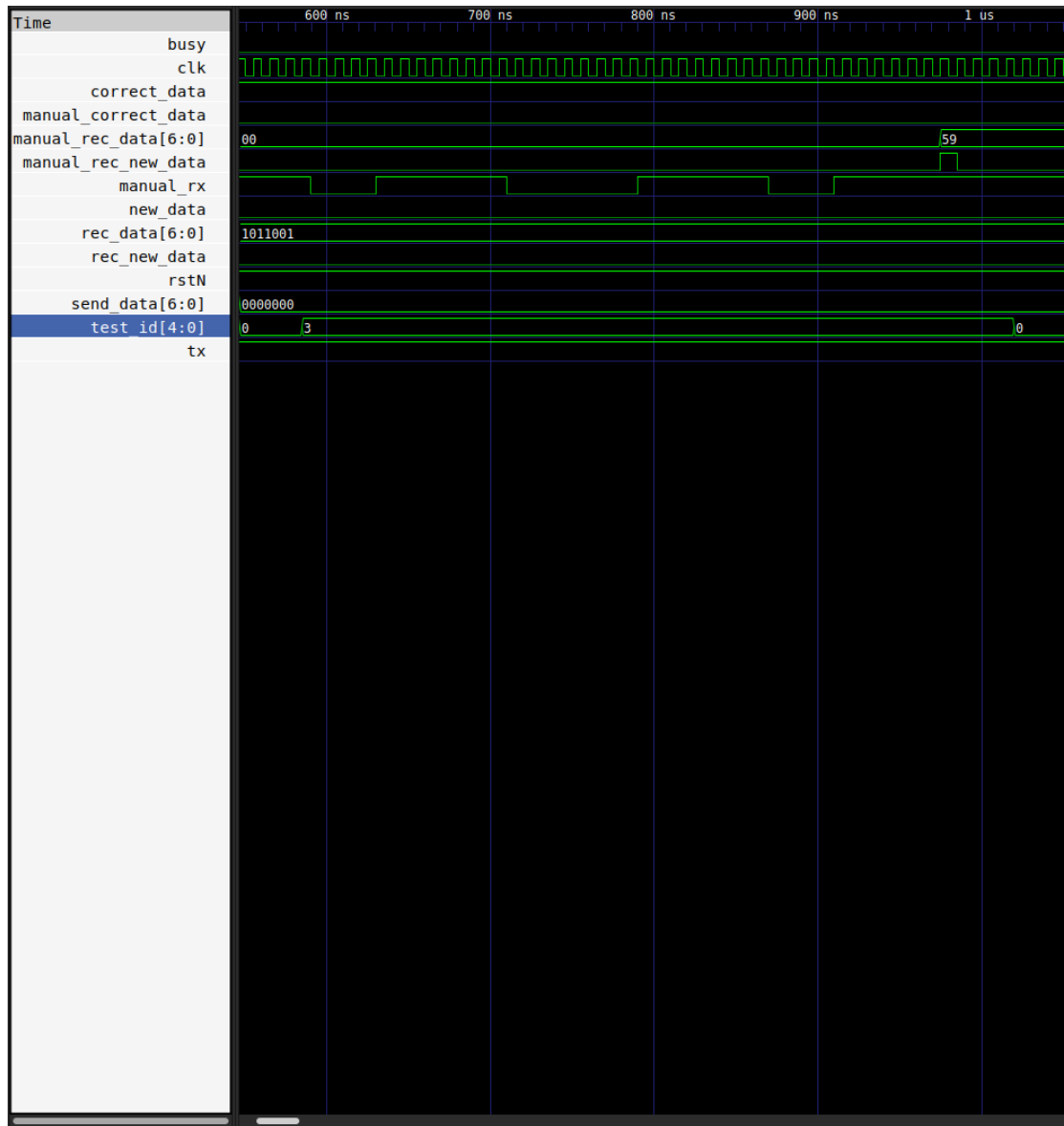
در این شکل آغاز انتقال با پالس `new_data` مشخص می‌شود و پس از آن `busy` فعال شده و `tx` به ترتیب بیت‌های قاب را حمل می‌کند. مقدار `send_data=1011001` در فرستنده لچ شده است و در پایان دریافت، `rec_data` همان مقدار را نشان می‌دهد. پالس کوتاه `rec_new_data` پایان فرآیند دریافت را مشخص می‌کند و فعال شدن `correct_data` نشان می‌دهد که قاب از نظر `parity` و `bit stop` معتبر بوده است.



شکل ۶: شکل موج منتخب برای آزمون دریافت صحیح و بررسی قالب قاب، `test_id=2`

۲.۹ شکل موج آزمون خطای parity، test_id=3

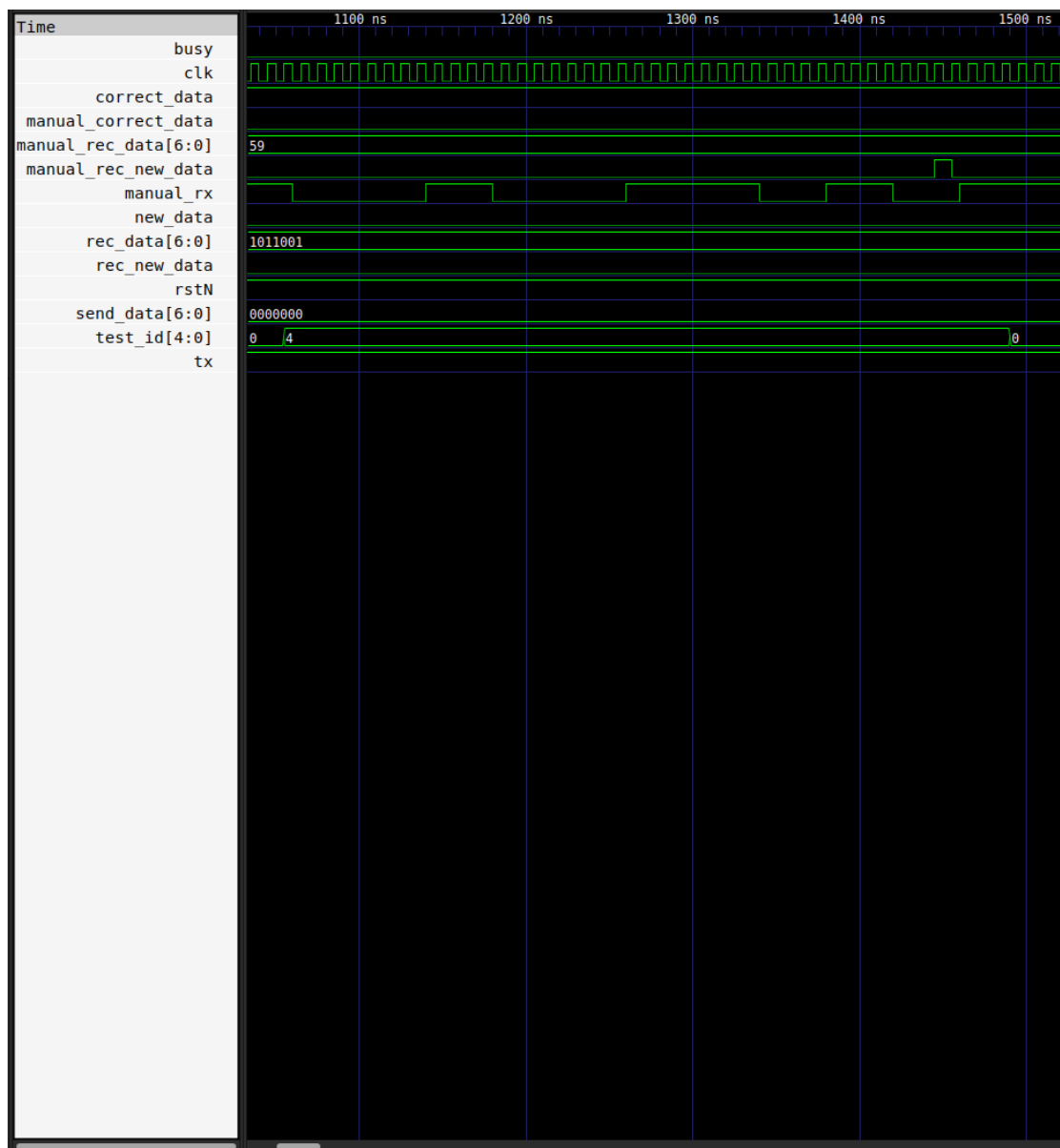
در این بازه قاب با داده صحیح ساخته می‌شود، اما parity به صورت عمدی نادرست است. گیرنده مستقل همچنان فرآیند دریافت را کامل می‌کند و داده را ذخیره می‌کند، ولی سیگنال صحت قاب فعال نمی‌شود. بنابراین شکل موج به طور هم‌زمان تفاوت میان «دریافت داده» و «معتبر بودن قاب» را نشان می‌دهد.



شکل ۷: شکل موج منتخب برای آزمون خطای parity، test_id=3

۳.۹ شکل موج آزمون خطای بیت توقف، test_id=4

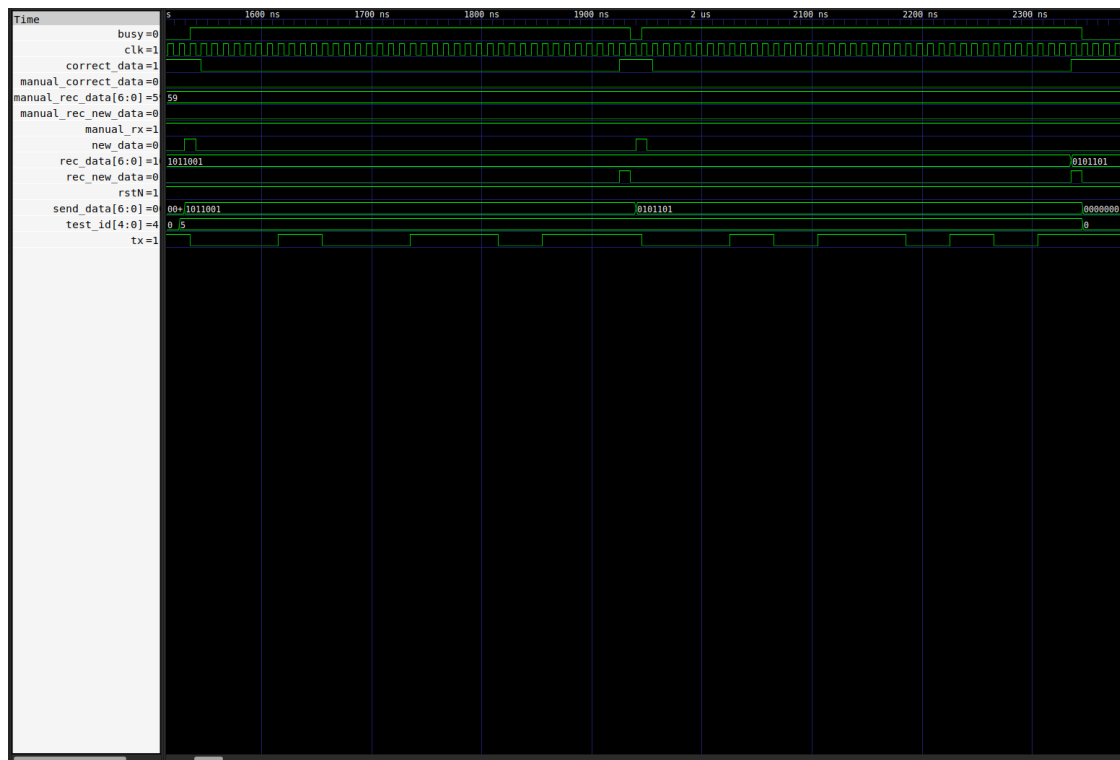
در این آزمون parity درست است، اما bit stop برابر صفر قرار داده شده است. به همین دلیل در پایان قاب، پالس دریافت ایجاد می‌شود ولی manual_correct_data صفر باقی می‌ماند. این شکل نشان می‌دهد که بررسی بیت توقف شرط مستقلی برای معتبر شناختن قاب است.



شکل ۸: شکل موج منتخب برای آزمون خطای بیت توقف، $\text{test_id}=4$

۴.۹ شکل موج آزمون ارسال چند قاب، $\text{test_id}=5$

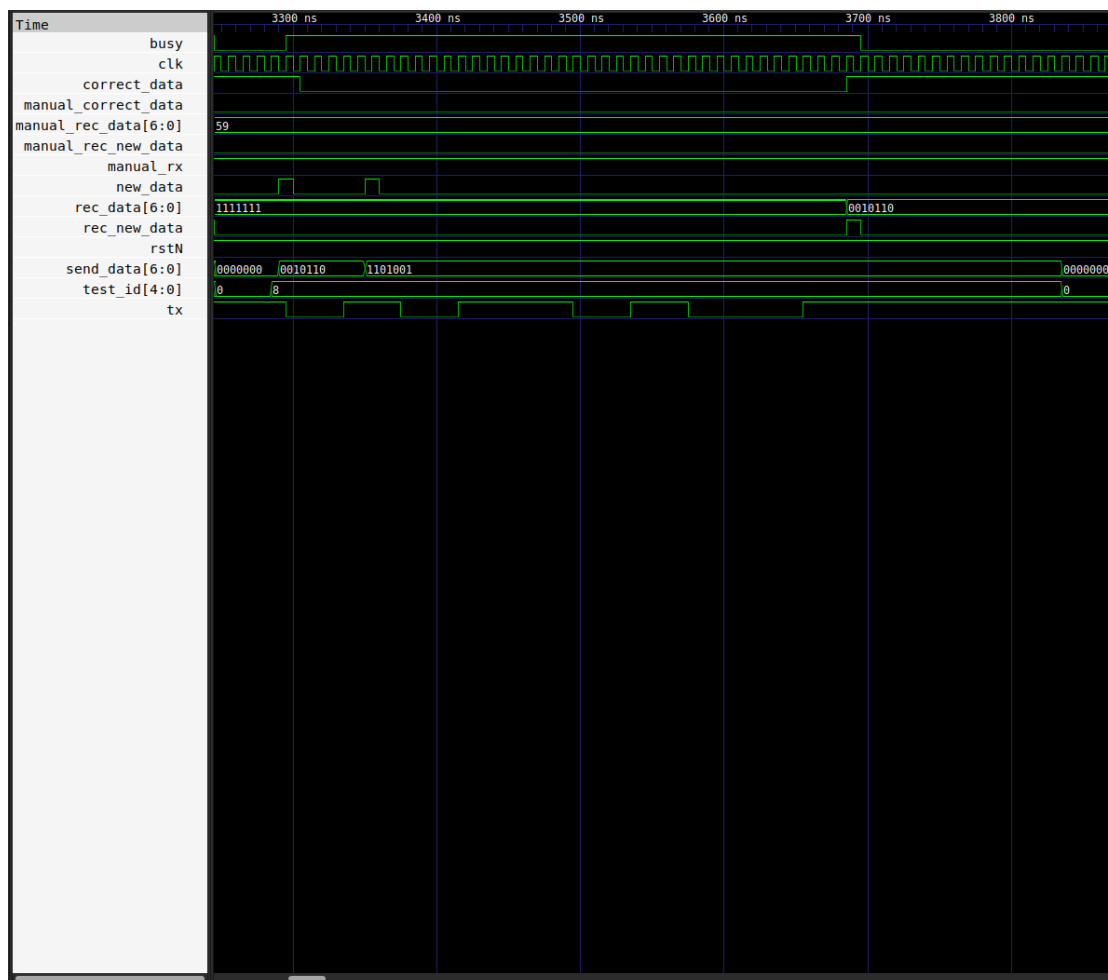
در این شکل دو قاب با داده‌های متفاوت پشت سرهم دیده می‌شوند. پس از پایان قاب نخست و بازگشت **busy** به صفر، قاب دوم آغاز می‌شود. در سمت گیرنده، هر داده به صورت مستقل در **rec_data** ظاهر شده و برای هر قاب یک پالس مجزا در **rec_new_data** ایجاد می‌شود. بنابراین زمان‌بندی پایان و آغاز قاب‌های متوالی صحیح است.



شکل ۹: شکل موج منتخب برای آزمون ارسال چند قاب پشت سرهم، test_id=5

۵.۹ شکل موج آزمون نادیده گرفتن new_data هنگام busy، test_id=8

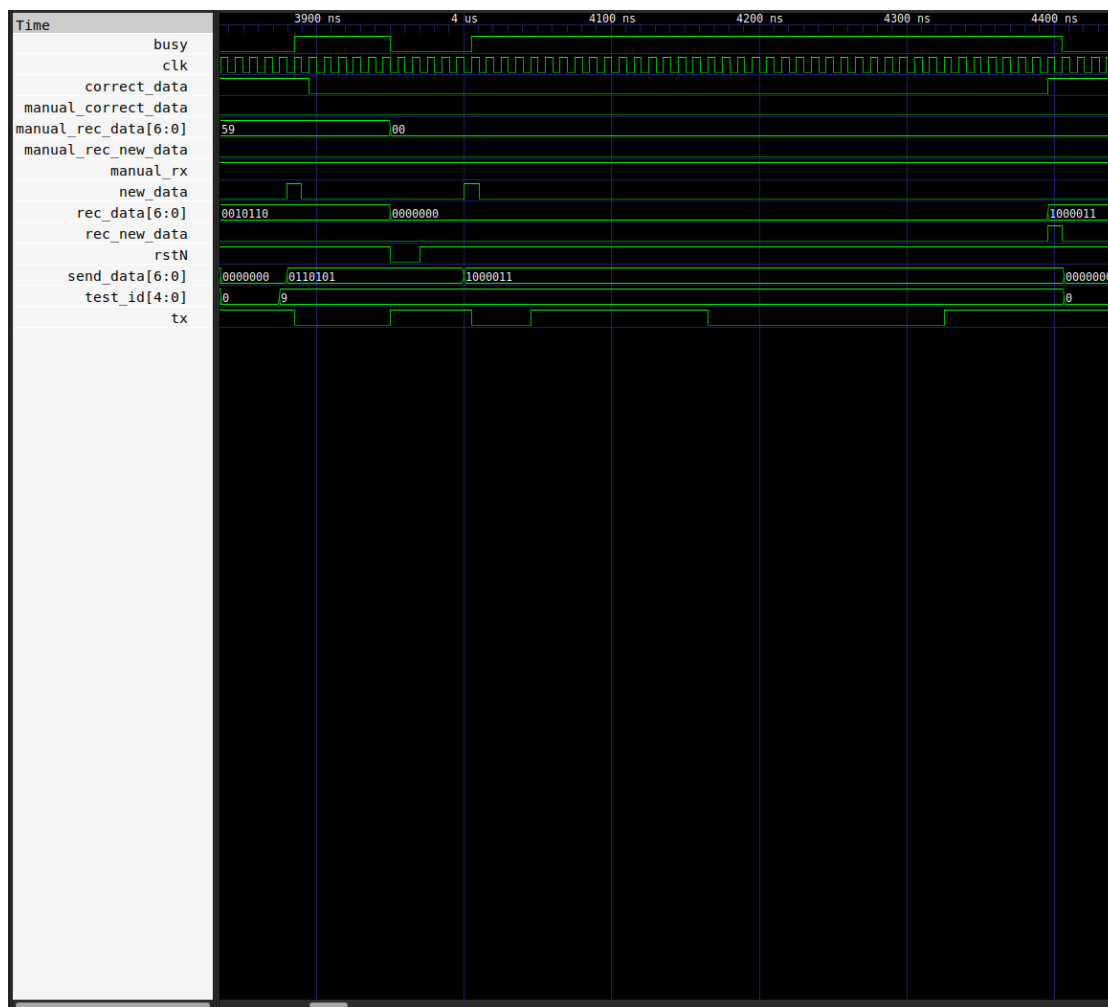
در این شکل در حالی که فرستنده در حال ارسال قاب اول است، داده ورودی تغییر کرده و یک درخواست جدید اعمال می شود. با وجود این تغییر، قاب جاری با همان داده اولیه ادامه پیدا می کند و پس از پایان آن تنها یک رویداد دریافت ایجاد می شود. این رفتار تأیید می کند که داده در ابتدای انتقال لچ شده و درخواست های جدید هنگام busy باعث خراب شدن یا صف شدن قاب دیگری نشده اند.



شکل ۱۰: شکل موج منتخب برای آزمون درخواست جدید هنگام مشغول بودن فرستنده، $test_id=8$

۶.۹ شکل موج آزمون بازنشانی میانه انتقال، $test_id=9$

در این بازه ابتدا یک قاب در حال انتقال است، سپس $rstN$ به صفر می‌رود. اثر بازنشانی ناهم‌زمان بلافاصله روی tx و $busy$ دیده می‌شود و قاب جاری بدون تولید رویداد دریافت معتبر متوقف می‌شود. پس از آزاد شدن بازنشانی، قاب جدید با داده 1000011 به صورت عادی منتقل شده و در گیرنده دریافت می‌شود؛ بنابراین مدار پس از یک قطع ناگهانی نیز به وضعیت صحیح برمی‌گردد.



شکل ۱۱: شکل موج منتخب برای آزمون بازنشانی ناهم‌زمان و بازیابی، $\text{test_id}=9$

۷.۹ توضیح کلی شکل موج‌ها

در تمامی شکل‌های بالا، سیگنال clk مبنای زمان‌بندی مدار است و با توجه به مقدار $\text{BIT_TICKS}=4$ هر بیت حدود ۴۰ نانوثانیه روی خط قرار می‌گیرد. سیگنال tx قاب را به ترتیب مورد انتظار حمل می‌کند، busy محدوده زمانی مشغول بودن فرستنده را مشخص می‌سازد، و rec_new_data پایان دریافت هر قاب را به صورت یک پالس کوتاه اعلام می‌کند. مقایسه send_data و rec_data نیز امکان بررسی مستقیم صحت بازگشت داده را فراهم می‌کند.

تصاویر حاضر به صورت منتخب از میان بازه‌های اجرای آزمون تهیه شده‌اند؛ در نتیجه ممکن است آزمون‌های ۱، ۶، ۷، ۱۰ و ۱۱ در تصویر مستقل نداشته باشند، ولی منطق و نتیجه همه آن‌ها در بخش سناریوها و خروجی نهایی فایل آزمون بررسی شده است.

۱۰ خروجی نهایی فایل آزمون

آخرین تصویر، خروجی متنی فایل آزمون است و نتیجه خودآزمایی تمام سناریوها را نشان می‌دهد.

```

VCD info: dumpfile uart_tests.vcd opened for output.
=====
Starting comprehensive UART testbench
BIT_TICKS=4, CLK_PERIOD=10 ns
=====
PASS: sender returns to idle immediately during reset
PASS: receiver status outputs clear during reset
PASS: manual receiver status outputs clear during reset

Test 1: reset and idle state
PASS: idle line is high and sender is not busy
PASS: no receive pulse is present while idle

Test 2: correct receive + frame format + busy timing
PASS: top received 1011001 correctly
PASS: busy clears only after stop bit and tx returns to idle
PASS: rec_new_data is a one-clock pulse

Test 3: parity error
PASS: manual receiver data=1011001 correct=0
PASS: manual rec_new_data is a one-clock pulse

Test 4: stop-bit error
PASS: manual receiver data=1011001 correct=0
PASS: manual rec_new_data is a one-clock pulse

Test 5: multiple back-to-back frames
PASS: top received 1011001 correctly
PASS: rec_new_data is a one-clock pulse
PASS: top received 0101101 correctly
PASS: rec_new_data is a one-clock pulse

Test 6: minimum payload 0000000
PASS: top received 0000000 correctly
PASS: rec_new_data is a one-clock pulse

Test 7: maximum payload 1111111 (valid parity = 1)
PASS: top received 1111111 correctly
PASS: rec_new_data is a one-clock pulse

Test 8: new_data while busy is ignored + input data is latched
PASS: top received 0010110 correctly
PASS: rec_new_data is a one-clock pulse
PASS: busy-time new_data created no extra received frame
PASS: sender returns to idle after ignored busy-time request

Test 9: asynchronous reset during transmission + recovery
PASS: mid-frame asynchronous reset immediately aborts sender
PASS: mid-frame reset clears receiver status
PASS: aborted frame does not produce a receive event
PASS: top received 1000011 correctly
PASS: rec_new_data is a one-clock pulse

Test 10: false start rejection + receiver recovery
PASS: short false-start pulse is rejected
PASS: false start is never marked as correct data
PASS: manual receiver data=0000001 correct=1
PASS: manual rec_new_data is a one-clock pulse

Test 11: exhaustive sweep of all 128 payload values
PASS: all 128 payload values received correctly

=====
UART TEST SUMMARY
Checks performed : 369
Errors found      : 0
ALL TESTS PASSED
VCD file          : uart_tests.vcd
=====
Tester.v:580: $finish called at 57615000 (1ps)

```

شکل ۱۲: خروجی نهایی فایل آزمون و خلاصه اجرای آزمون‌های UART

مطابق این خروجی، همه آزمون‌های اصلی با پیام‌های PASS پشت‌سر گذاشته شده‌اند. در آزمون آخر نیز تمام ۱۲۸ مقدار ممکن داده هفت‌بیتی بدون خطا دریافت شده‌اند. همچنین در خلاصه نهایی، تعداد کل بررسی‌ها برابر ۳۶۹ و تعداد خطاها برابر صفر گزارش شده است و در نتیجه عبارت ALL TESTS PASSED نمایش داده می‌شود.

۱۱ تحلیل نتایج و جمع‌بندی

نتایج این آزمایش سه بخش اصلی طراحی را تأیید می‌کند. نخست، منطق فرستنده و گیرنده مطابق قالب تعریف‌شده عمل می‌کند؛ یعنی بیت شروع، parity هفت بیت داده و بیت توقف با ترتیب و زمان‌بندی صحیح روی خط قرار می‌گیرند و گیرنده

نیز همان ترتیب را دنبال می‌کند. دوم، اعتبار قاب به‌صورت مستقل از دریافت داده بررسی می‌شود و خطا در parity یا stop bit باعث صفرشدن سیگنال صحت می‌شود. سوم، فایل آزمون فقط رفتار عادی مدار را بررسی نمی‌کند و حالت‌های مرزی مؤثر بر عملکرد واقعی مدار را نیز پوشش می‌دهد.

وجود گیرنده مستقل برای تزریق خطا باعث شده است آزمون‌های parity و bit stop واقعاً مستقل از منطق فرستنده باشند. بررسی مستقیم بیت‌های tx نیز از آن جلوگیری می‌کند که در یک بازگشت مستقیم، اگر فرستنده و گیرنده یک خطای مشترک در ترتیب بیت داشته باشند، نتیجه به‌اشتباه صحیح گزارش شود. همچنین بررسی یک‌کلاک بودن rec_new_data تضمین می‌کند که رویداد دریافت به‌صورت یک پالس قابل استفاده برای منطق بالاتر تولید می‌شود.

در نهایت، موفقیت همه ۳۶۹ بررسی و صفر بودن تعداد خطاها، همراه با عبور کامل از پیمایش ۱۲۸ مقدار داده هفت‌بیتی، نشان می‌دهد که طراحی ارائه‌شده برای سناریوهای بررسی‌شده پایدار و قابل‌اعتماد است. بنابراین می‌توان نتیجه گرفت که ساختار فرستنده، گیرنده، اتصال سطح بالا و فایل آزمون جامع، تمام خواسته‌های اصلی آزمایش را برآورده کرده‌اند.

۱۲ فایل‌های تحویلی

مطابق صورت آزمایش، فایل‌های اصلی همراه گزارش عبارت‌اند از:

- UARTSender.v: طراحی فرستنده و ماشین حالت ارسال؛
- UARTReceiver.v: طراحی گیرنده، دریافت داده و بررسی parity و stop bit؛
- UARTTop.v: اتصال فرستنده به گیرنده در قالب بازگشت مستقیم؛
- Tester.v: فایل آزمون جامع برای تولید ساعت، اعمال بازنشانی، تولید داده، تزریق خطا، بررسی خروجی‌ها و اجرای آزمون جامع ۱۲۸ حالتی؛
- uart_tests.vcd: فایل حاصل از ثبت تغییرات سیگنال‌ها در اجرای شبیه‌سازی.